

Revenue Optimiser

A defensible, data-grounded, AI-assisted pricing engine for rental operators. Decisions, not guesses.

Revenue Optimiser

Drive247's pricing intelligence layer

A specification for building a defensible, data-grounded, AI-assisted pricing engine for rental operators.

Table of Contents

1. Executive Summary
 2. Product Positioning
 3. The Legitimacy Stack — 10 Principles That Make This Real
 4. User Personas
 5. Phased Rollout
 6. User Journeys
 7. Feature Catalogue
 8. UI Screens
 9. Data Architecture
 10. Edge Functions
 11. Algorithm Specification
 12. AI Integration
 13. Safety & Trust Mechanisms
 14. What to Show / What to Hide
 15. Monitoring & Telemetry
 16. Subscription Tier & Pricing
 17. Risks & Mitigations
 18. Success Metrics
 19. Build Checklist
 20. Appendix
-

1. Executive Summary

What we are building. A pricing intelligence layer inside Drive247 — branded **Revenue Optimiser** — that helps rental operators set the right price for each vehicle, at the right time, based on their own fleet's demand, utilisation, idle days, and conversion history.

Why. Most rental operators set prices once and forget. The result is consistent under-pricing in peak periods and over-pricing on idle cars. Independent industry data suggests dynamic pricing typically lifts

revenue by 8–15% in the rental vertical when done responsibly. Operators have asked for this. Drive247 has the data to deliver it.

How it is different. Most “AI pricing” products in this category are GPT wrappers with no math behind them. Revenue Optimiser is built around a deterministic pricing model — price elasticity, demand/supply scoring, conversion data — with GPT used only to explain the recommendation in plain English. Every recommendation is backed by a chart, a confidence level, and an outcome that gets measured 14 days later.

The promise to the operator.

Drive247’s Revenue Optimiser shows you the best price for each vehicle, explains why, lets you approve or set safe rules, and proves the revenue lift on your own data.

Target. Growth-tier tenants with 10+ vehicles and 60+ days of booking history. Below that, the feature operates in observation-only mode.

Build estimate. 6–8 engineering weeks across four phases. MVP (Phases 0–2) shippable in 4 weeks.

2. Product Positioning

Name

Revenue Optimiser. Not “AI Pricing.” Operators care about revenue, not buzzwords. “AI” goes in the explainer copy, never in the product name.

Tagline options

- “Price smarter. Fill more cars. Prove the lift.”
- “The pricing layer your fleet has been missing.”
- “Decisions, not guesses.”

One-liner for the sales deck

Revenue Optimiser watches every vehicle in your fleet — demand, idle days, utilisation, enquiries, conversion — and tells you the best price to charge, explains why, and measures the impact. You stay in control.

What it is NOT

- Not autopilot-only. Operators stay in the driver’s seat by default.
- Not a market-data product. We don’t scrape competitors in v1.

- Not magic. Recommendations are grounded in the tenant's own history.

Where it sits in the product

Top-level sidebar item in the portal, under a new **Revenue** group (which we can grow over time with related products: discounts, promotions, loyalty). Initially the group contains only Revenue Optimiser.

Why this is defensible

The moat is not the algorithm. The moat is:

1. **Drive247 owns the booking, the customer, and the payment.** Competitors selling pricing tools have no booking context. We do.
 2. **Outcome data accumulates.** Every applied recommendation feeds back into the model. The more tenants use it, the better it gets — for them specifically.
 3. **Integration with Lead Hub.** When the recommendation is “drop price,” we don’t just drop — we tell the operator which waitlisted enquiries to offer it to.
-

3. The Legitimacy Stack — 10 Principles That Make This Real

These are the rules. If we cut corners on any of them, the feature becomes the kind of shallow “AI pricing” we are trying to differentiate from. Every engineering and product decision should be checked against this list.

1. Math first, AI second

The recommended price is computed by a deterministic statistical model (elasticity + demand/supply scoring). GPT is used only to translate the model's output into a plain-English explanation. We never ask GPT to invent a price.

2. Show your work, always

Every recommendation displays the specific data points behind it: bookings in last 30 days, utilisation %, active enquiries, idle days, conversion rate. No opaque numbers. Operators must be able to argue with the recommendation.

3. Outcome feedback loops from day one

Every applied recommendation is tagged. 14 days later we measure: did bookings come? Did revenue move? Net impact in dollars. This becomes the operator's monthly proof point (“Revenue Optimiser added \$4,820 this quarter”) and feeds back into our elasticity model.

4. Confidence intervals, never point estimates

We show a range, not a single number. “Recommended: **\$310–\$340**, confidence: **High** (based on 47 bookings in last 90 days).” If sample size is low, confidence drops, and we say so.

5. Backtesting before the operator commits

On day one, before recommending anything, we replay the tenant’s last 6 months of bookings against what the engine *would have* recommended, and show the operator the projected lift. If we can’t beat their actuals on their own data, we don’t ship to them.

6. Calibration period for new tenants

First 30 days for a new tenant = observation mode only. The engine collects data, surfaces simple insights, but makes no price recommendations. This protects new tenants from premature advice.

7. Hard safety rails

- Max swing per recommendation: $\pm 15\%$ by default (operator-configurable)
- Cost floor per vehicle (never recommend below operator-set break-even)
- Manager approval threshold (auto-pause recommendations above \$X)
- Auto-pause autopilot if utilisation drops $>20\%$ within 7 days of applying

8. A/B testing in autopilot mode

Autopilot does not change all vehicles at once. It applies changes to a control group, measures for 14 days, then rolls out winners. Phase 3.

9. Honest scope

We tell operators explicitly: “Revenue Optimiser learns from *your* fleet. It does not predict external events like concerts or weather (yet).” No overpromising.

10. Cost transparency

OpenAI usage is logged per tenant and visible to Drive247 super-admin. Tenants don’t see raw cost but the feature is priced into the Growth tier in a way that comfortably covers compute even at high recommendation volume.

4. User Personas

Sarah — Operator / Tenant Admin

- Runs City Wheels, 45 vehicles, mixed economy + SUV + Tesla
- Sets prices manually, updates them maybe quarterly

- Knows her market intuitively but can't track every vehicle daily
- Wants: "Tell me what to charge and why. I'll decide if I trust it."
- Mistrust trigger: a recommendation that obviously makes no sense

Mike — Manager (limited permissions)

- Mike has `manager` role with access to the Revenue Optimiser tab in `viewer` mode
- Can see recommendations but not apply them; must escalate to Sarah
- For larger tenants, Sarah may grant `editor` access on the tab

Drive247 Ops (super admin)

- Monitors aggregate model performance across all tenants
- Sees OpenAI cost per tenant
- Can flag/suppress a recommendation manually if it looks unsafe
- Receives anomaly alerts (e.g., utilisation crash after applying)

5. Phased Rollout

Phase	What ships	Duration	Risk
0 — Foundation	Aggregation views, backtest engine, data quality checks	Week 1	Low
1 — Insights	Observation mode, no recommendations, daily summary insights	Weeks 2–3	Low
2 — Recommendations	Per-vehicle recommendations with apply/dismiss, audit log	Weeks 4–5	Medium
3 — Rules & Autopilot	Tenant-defined rules, autopilot with A/B testing	Weeks 6–7	High — needs Phase 2 trust
4 — Lead Hub Integration	"Send offer to matching enquiries" flow	Week 8	Medium

Each phase must hit acceptance criteria before the next starts. Phase 3 should not ship until at least 5 tenants have run Phase 2 for 30 days with measured positive uplift.

6. User Journeys

Five concrete walkthroughs, each with the screen flow and what data the operator sees.

Journey A — First-time setup (Sarah, City Wheels)

Entry point: Sarah upgrades to Growth tier. A new “Revenue Optimiser” item appears in her portal sidebar with a “NEW” badge.

1. Sarah clicks Revenue Optimiser. She lands on the **Welcome screen**.
2. The screen says:
 - “Revenue Optimiser learns from your fleet’s history to suggest the best price for each vehicle.”
 - “Before we recommend anything, let’s look at your last 6 months.”
 - Button: **Run Backtest**.
3. Sarah clicks. The backtest takes ~30 seconds (edge function). During the wait, she sees a progress bar and 3 bullet points explaining what’s happening: “Loading your bookings”, “Modelling demand”, “Calculating projected lift”.
4. Backtest report loads:
 - **“If you had used Revenue Optimiser for the last 6 months, projected revenue: \$182,400 vs. actual \$171,250. Estimated lift: +6.5% (\$11,150).”**
 - Chart: monthly revenue actual vs. projected.
 - Per-vehicle table: top 10 vehicles with the biggest projected lift.
 - Caveats clearly listed: “Backtest assumes historical demand. Actual results depend on market conditions and how often you apply recommendations.”
5. Sarah is intrigued but not convinced. She clicks **Enable Insights Mode** (lower-risk option, shown before “Enable Recommendations”).
6. Confirmation: “Insights Mode is on. We’ll collect data and surface observations. You won’t see price recommendations until you switch to Recommendations Mode.”
7. Sidebar now shows a small badge: “Insights active.”

Why this matters: The backtest is the trust anchor. Operators who would never enable a black-box pricing tool will enable one that already showed them money they left on the table.

Journey B — Daily check-in and applying a recommendation

Entry point: Sarah enabled Recommendations Mode two weeks ago. She opens the portal Monday at 9am.

1. The portal dashboard top strip shows: **“Revenue Optimiser: 4 new opportunities worth +\$620/mo projected”** with a “Review” button.
2. Sarah clicks Review. She lands on the **Recommendations** page.

3. She sees 4 cards, each one a recommendation. They are sorted by projected revenue impact (highest first).
4. Top card:

```
Toyota Corolla · Plate ABC-123          [HIGH CONFIDENCE]
Current weekly: $300          Recommended: $315-$330
Projected lift: +$95/month    · Apply window: next 14 days
Why now:
  • 87% of similar vehicles in your fleet booked at $300+
  • 4 active enquiries match this vehicle (next 14 days)
  • Conversion rate at $300 last 90d: 76%
  • Vehicle booked 28 of last 30 days (93% utilisation)
Risk: At $325, projected conversion: 68% (-8 pp).
      Net effect: positive in 90% of simulated scenarios.
[ Apply $325 ] [ Apply $315 ] [ Custom... ] [ Dismiss ]
[ Snooze 7 days ]
```

5. Sarah hovers on “Apply \$325.” A tooltip explains: “Updates weekly rate from \$300 to \$325. We’ll check the outcome in 14 days.”
6. She clicks Apply. Confirmation dialog shows the change, the audit trail message (“This change is logged. You can revert anytime.”), and a checkbox: “Notify me of outcome in 14 days.” (Default on.)
7. She confirms. The card animates to “Applied” state and stays visible at the bottom of the page for 7 days in a “Recently Applied” section.

What she does NOT see:

- The internal multiplier coefficients
- The GPT prompt
- Other tenants’ data
- Raw OpenAI cost

Journey C — Idle vehicle + send offer to enquiries (Phase 4)

Entry point: Notification email arrives at 8am — “1 idle vehicle, 4 waiting enquiries.”

1. Sarah clicks through to the Revenue Optimiser page. The top recommendation is now a **combined card** because the system detected both an idle vehicle and matching enquiries.


```
Nissan Altima · Plate XYZ-789      [IDLE 6 DAYS]

Idle since: 16 May · Last booked: 10 May

Two-part recommendation:

1. Adjust weekly rate $300 → $270 for next 7 days
   Projected: 1-2 bookings recovered in 14 days

2. Send offer to 4 waitlisted enquiries who match
   Enquiries received in last 21 days for similar vehicle

Matching enquiries:
• Aisha B. – wants 19-26 May · phone + email
• James L. – wants 22-28 May · email only
• Priya K. – wants 24-31 May · phone + email
• Tom M.   – wants 20-25 May · phone + email

[ Apply price + send offers ] [ Send offers only ]
[ Apply price only ] [ Dismiss ]
```

2. Sarah clicks “Apply price + send offers.”

3. A second screen previews the offer message (uses tenant’s existing comms templates):

```
Good news, Aisha – a Nissan Altima just became available for your dates
(19-26 May). Special weekly rate of $270 if you book by tomorrow. Reply
YES or click here to confirm.
```

```
[ Edit message ] [ Send to 4 enquiries ] [ Cancel ]
```

4. Sarah clicks Send. We dispatch via existing email/SMS infrastructure.

5. Within hours, 2 enquiries reply. Drive247’s Lead Hub already tracks this; the recommendation card now shows: “2 of 4 contacted leads have engaged.”

This is the moment where the feature pays for itself. Pricing alone is useful. Pricing tied to lead activation is unbeatable.

Journey D — Setting up Autopilot rules (Phase 3)

Entry point: Sarah has been applying recommendations manually for 60+ days. Her Revenue Optimiser dashboard shows: “**+\$4,820 added in last 90 days**, 38 of 41 recommendations applied, 87% had positive measured outcome.”

1. She clicks the **Autopilot** tab (locked behind a “Try Autopilot” CTA until eligibility is met).
2. The Autopilot setup wizard opens. Step 1: choose scope.

Which vehicles should Autopilot manage?

- ☒ Economy vehicles only (← Recommended start)
- ☐ Economy + SUV
- ☐ All vehicles
- ☐ Custom selection

 We recommend starting with one category. You can expand later.

3. Step 2: set safety rails.

Safety rails:

Max swing per change: ±15% [slider 5–30%]
Cost floor per vehicle: Enabled [edit floors]
Weekend max increase: +20% [slider 0–40%]
Auto-pause if utilisation drops: Enabled
Require approval above: \$40/day change

 Autopilot will never breach these limits.

4. Step 3: review.

You're enabling Autopilot for:

- 12 economy vehicles
- With ±15% max swing
- Daily review at 7am
- Weekly summary email
- Audit log retained 12 months

Estimated impact: \$1,400–\$2,100/mo

[Enable Autopilot] [Back] [Cancel]

5. Sarah confirms. Autopilot is now on for economy vehicles. Each morning at 7am, the system applies any in-bound recommendations and sends a summary at 9am.

Journey E — Backtest review (skeptical operator)

Entry point: A new tenant, “Hertz Replacement Ltd,” is wary of AI tools. Sales asks if we can show real numbers.

1. Drive247 ops generates an on-demand backtest from the admin tools.
2. The output is a shareable PDF (auto-generated):
 - Tenant name, fleet size, backtest period
 - Actual vs. projected revenue, month by month
 - Per-vehicle uplift table
 - Confidence band
 - Caveats
3. Sales emails the PDF. The operator sees: “On your own bookings, in your own market, this would have added \$32K in 6 months.”
4. Conversion follows.

This is a sales asset, not just a feature.

7. Feature Catalogue

Grouped by surface area.

Portal screens

- Revenue Optimiser welcome / onboarding
- Backtest report
- Recommendations list (default landing once enabled)
- Recommendation detail / explainability drawer
- Vehicle detail: Smart Pricing panel embedded
- Autopilot configuration
- Rules editor (min/max per vehicle or category)
- Outcome tracker (applied recommendations and their measured results)
- Change history / audit log
- Settings: notifications, autopilot, cost floors

Admin (super-admin) screens

- Per-tenant Revenue Optimiser health dashboard
- OpenAI cost per tenant
- Anomaly alerts inbox
- Force-suppress a recommendation
- Aggregate model performance

Notifications

- Daily morning summary (email)

- Outcome notification 14 days after apply
- Anomaly alert if applied recommendation caused utilisation drop
- Weekly digest

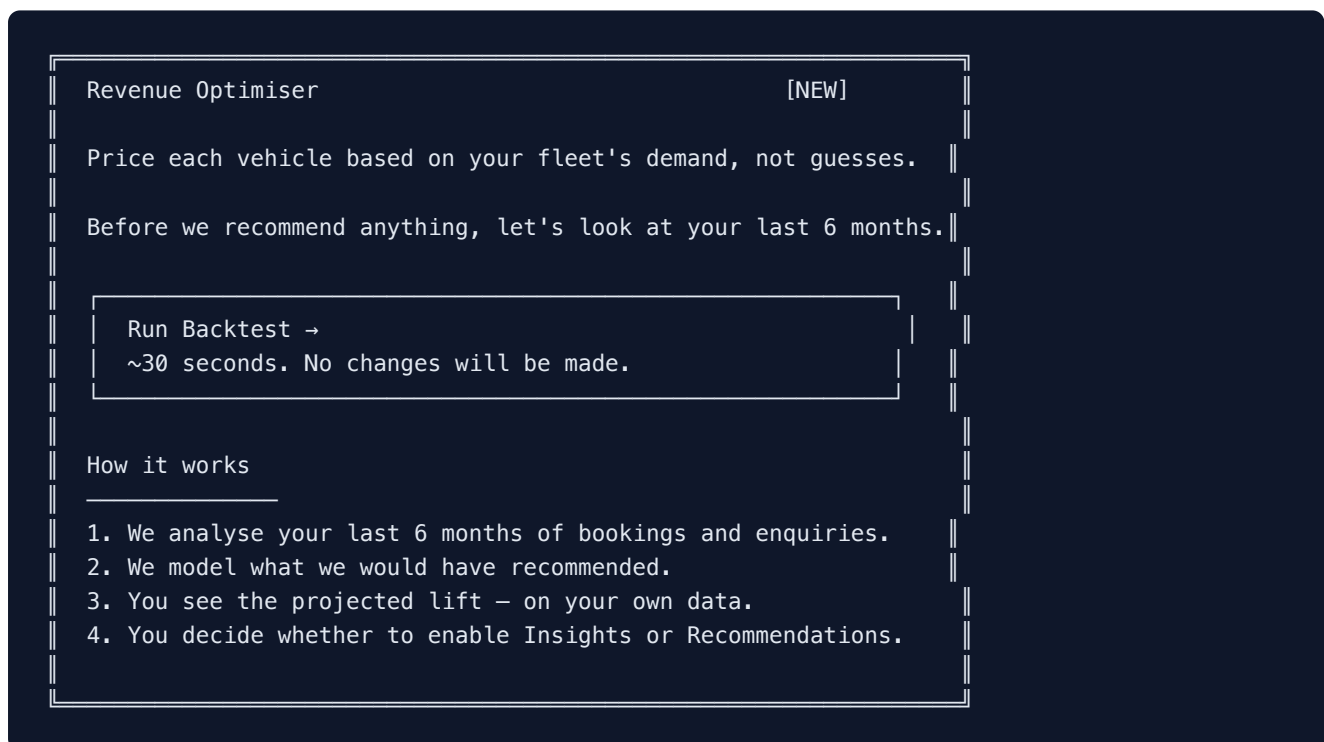
Edge functions / cron jobs

- Daily recommendation generation
- Hourly stats refresh
- Daily outcome measurement
- Weekly model accuracy report
- On-demand backtest

8. UI Screens

ASCII mockups. Final designs follow Drive247 portal design system (DM Sans, indigo accent #6366f1, flat cards with 1px borders, no shadows).

8.1 Welcome screen (first visit)



8.2 Backtest report

Backtest Report
Period: Nov 2025 – Apr 2026

City Wheels · 45 cars

Projected lift

+6.5% (+\$11,150)

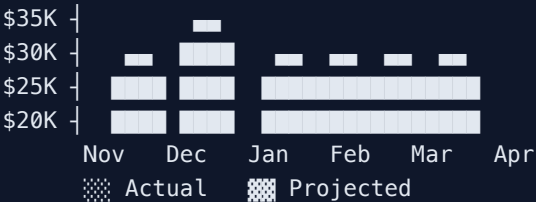
Actual: \$171,250
Projected: \$182,400

Confidence:

High

Based on 612 booking events

Revenue by month (actual vs. projected)



Top vehicles by projected lift

Vehicle	Actual	Projected	Lift	
Toyota Corolla ABC	\$3,200	\$3,540	+\$340	+11%
Tesla Model 3 XYZ	\$5,800	\$6,310	+\$510	+ 9%
Nissan Altima DEF	\$2,900	\$3,180	+\$280	+10%
... 42 more				

Caveats

- Backtest assumes historical demand patterns.
- Real-world results depend on how often you apply recs.
- New vehicles (<60d history) excluded from this estimate.

[Enable Insights Mode] [Enable Recommendations] [Later]

8.3 Recommendations list

Revenue Optimiser > Recommendations

Recommendations ON

Summary: 4 new · +\$620/mo projected · 89% historical accuracy

[Sort: Impact ▼] [Filter: All ▼] [Status: Pending ▼]

Toyota Corolla ABC-123

HIGH

+\$95/mo

Weekly: \$300 → \$325

▶ 87% similar booked · 4 enquiries · 76% conversion

[Apply] [Details] [Dismiss]

Nissan Altima XYZ-789

MEDIUM

+\$210/mo

IDLE 6 days · Weekly: \$300 → \$270 + send offers (4)

▶ 0 bookings in 6 days · 4 matching enquiries

[Apply + Send] [Apply Only] [Details] [Dismiss]

Tesla Model 3 LMN-456

HIGH

+\$240/mo

Weekend daily: \$95 → \$108

▶ Friday demand +34% MoM · 3 weeks fully booked

[Apply] [Details] [Dismiss]

Ford Transit GHI-321

LOW

+\$75/mo

Monthly: \$1,400 → \$1,470

⚠ Low confidence – only 8 bookings in last 90 days

[Apply] [Details] [Dismiss]

Recently Applied (last 7 days) ▼

8.4 Recommendation detail drawer

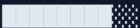
Recommendation Detail

[x]

Toyota Corolla · ABC-123 · Economy

Recommended weekly rate: \$315–\$330 (suggested \$325)

Current: \$300

Confidence:  HIGH

Based on 47 bookings in last 90 days for this vehicle.

What we observed:

Utilisation (last 30d)	93% ▲ vs fleet 78%
Conversion rate at \$300	76%
Bookings velocity (last 14d)	+8% vs prior 14d
Active enquiries matching	4
Idle days in last 30	2
Days till next booking	0 (currently booked)

Elasticity model:

Price	Projected bookings/mo	Projected revenue/mo
\$290	4.3	\$1,247
\$300	4.1 (current)	\$1,230
\$315	3.9	\$1,229
\$325 ✓	3.8	\$1,235 ← recommended
\$340	3.4	\$1,156

Risk scenarios:

- 90% chance: positive revenue impact
- 7% chance: neutral (within ±\$30/mo)
- 3% chance: negative – reverts triggered automatically

Plain explanation:

This vehicle is one of your most-rented. It's been booked 93% of the last 30 days, well above your fleet average. Four active enquiries are looking for similar dates. At \$325 we expect a small dip in conversion but a net revenue gain.

[Apply \$325] [Apply custom...] [Dismiss] [Snooze 7d]

8.5 Vehicle Smart Pricing panel (on vehicle detail page)

Smart Pricing

Powered by R0

Active recommendation:

Weekly: \$300 → \$325HIGH confidence+\$95/mo

[\[Apply \]](#) [\[Details \]](#) [\[Dismiss \]](#)

Last applied: 6 May (Daily \$50 → \$54, +\$28/mo measured)

90-day pricing performance

Revenue	\$3,540	(above fleet avg)
Idle days	4	
Conversion	76%	(at average price \$302)

[\[View full history \]](#)

8.6 Autopilot configuration

Autopilot [OFF / ON]

Status: Eligible – you have 62 days of measured outcomes and 87% historical accuracy.

Scope

- ☒ Economy vehicles (12)
- ☐ Economy + SUV (24)
- ☐ All vehicles (45)
- ☐ Custom...

Safety rails

Max swing per change  ±15%

Cost floor ☒ Enabled [edit floors]

Weekend max increase  +20%

Auto-pause utilisation ☒ Drop >20% in 7d

Approval required above [\$40] per change

A/B testing

☒ Split-test before rollout (14-day window)

Notifications

☒ Daily morning summary

☒ Outcome notifications (14d after apply)

☒ Anomaly alerts

Estimated impact: \$1,400–\$2,100/mo

[Enable Autopilot] [Cancel]

8.7 Outcome tracker (proves the lift)

Revenue Optimiser > Outcomes

Last 90 days

Applied: 41 recommendations
Positive: 36 (87%)
Neutral: 3 (7%)
Reverted: 2 (5%)

Measured revenue impact: +\$4,820
Confidence in measurement: High

Recent outcomes:

Vehicle	Change	14d outcome	Net
Corolla ABC	Weekly +\$25	Bookings 4 → 4	+\$92
Altima XYZ	Weekly -\$30	Bookings 0 → 2	+\$540
Tesla 3 LMN	Weekend +\$13	Bookings 6 → 5	+\$48
Transit GHI	Monthly +\$70	Bookings 1 → 0	-\$140
...			

[Export CSV] [Share report]

8.8 Daily summary email (text version)

From: Drive247 Revenue Optimiser
Subject: Your fleet this morning – 4 opportunities worth \$620/mo

Good morning Sarah,

Revenue Optimiser found 4 pricing opportunities for City Wheels.

- Toyota Corolla ABC-123 – Weekly \$300 → \$325 (+\$95/mo)
- Nissan Altima XYZ-789 – IDLE 6d, drop to \$270 + offer to 4 enquiries (+\$210/mo)
- Tesla Model 3 LMN-456 – Weekend +\$13/day (+\$240/mo)
- Ford Transit GHI-321 – Monthly +\$70 (+\$75/mo, low confidence)

Review and apply: <https://citywheels.portal.drive-247.com/revenue>

This week's outcomes:

- ✓ 3 of 3 recommendations applied 14 days ago were positive
- ✓ Measured impact so far this month: +\$1,180

– Drive247 Revenue Optimiser

9. Data Architecture

All schema lives in Supabase. Follow project conventions: `set_updated_at()` triggers, `get_user_tenant_id()` for RLS, snake_case columns, UUID PKs.

9.1 New tables

`revenue_optimiser_settings`

One row per tenant.

```
CREATE TABLE revenue_optimiser_settings (  
  tenant_id UUID PRIMARY KEY REFERENCES tenants(id) ON DELETE CASCADE,  
  enabled BOOLEAN DEFAULT FALSE,  
  mode TEXT NOT NULL DEFAULT 'observation'  
  CHECK (mode IN ('observation', 'recommendations', 'autopilot')),  
  calibration_complete BOOLEAN DEFAULT FALSE,  
  calibration_started_at TIMESTAMPTZ,  
  backtest_completed_at TIMESTAMPTZ,  
  backtest_projected_lift_percent NUMERIC(5,2),  
  backtest_projected_lift_amount NUMERIC(12,2),  
  
  -- safety rails  
  max_swing_percent NUMERIC(5,2) DEFAULT 15.0,  
  weekend_max_increase_percent NUMERIC(5,2) DEFAULT 20.0,  
  cost_floor_enabled BOOLEAN DEFAULT TRUE,  
  require_approval_above_amount NUMERIC(10,2),  
  auto_pause_on_utilization_drop BOOLEAN DEFAULT TRUE,  
  auto_pause_threshold_percent NUMERIC(5,2) DEFAULT 20.0,  
  
  -- notifications  
  notify_daily_summary BOOLEAN DEFAULT TRUE,  
  notify_outcome BOOLEAN DEFAULT TRUE,  
  notify_anomalies BOOLEAN DEFAULT TRUE,  
  
  created_at TIMESTAMPTZ DEFAULT now(),  
  updated_at TIMESTAMPTZ DEFAULT now()  
);
```

`pricing_recommendations`

One row per recommendation generated.

```

CREATE TABLE pricing_recommendations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  vehicle_id UUID NOT NULL REFERENCES vehicles(id) ON DELETE CASCADE,

  tier TEXT NOT NULL
    CHECK (tier IN ('daily', 'weekly', 'monthly', 'weekend_daily')),

  current_price NUMERIC(10,2) NOT NULL,
  recommended_price NUMERIC(10,2) NOT NULL,
  recommended_range_low NUMERIC(10,2) NOT NULL,
  recommended_range_high NUMERIC(10,2) NOT NULL,

  confidence TEXT NOT NULL
    CHECK (confidence IN ('low', 'medium', 'high')),
  confidence_score NUMERIC(5,2) NOT NULL,    -- 0-100

  projected_revenue_delta_monthly NUMERIC(10,2),

  -- structured reasoning
  reasons JSONB NOT NULL,                    -- array of {code, label, value, weight}
  data_points JSONB NOT NULL,                -- raw stats snapshot
  elasticity_curve JSONB,                    -- price/qty pairs for chart

  ai_explanation TEXT,                        -- GPT-generated plain English
  ai_model TEXT,                             -- e.g. gpt-4o-mini
  ai_tokens_total INTEGER,

  status TEXT NOT NULL DEFAULT 'pending'
    CHECK (status IN ('pending', 'applied', 'dismissed', 'snoozed',
                      'expired', 'reverted', 'superseded')),
  applied_at TIMESTAMPTZ,
  applied_by UUID REFERENCES app_users(id),
  applied_price NUMERIC(10,2),               -- may differ from recommended (custom)
  applied_source TEXT
    CHECK (applied_source IN ('manual', 'autopilot')),

  dismissed_at TIMESTAMPTZ,
  dismissed_by UUID REFERENCES app_users(id),
  dismiss_reason TEXT,
  snoozed_until TIMESTAMPTZ,

  reverted_at TIMESTAMPTZ,
  reverted_by UUID REFERENCES app_users(id),
  revert_reason TEXT,

  expires_at TIMESTAMPTZ NOT NULL,          -- recommendations decay; new one supersedes
  generation_run_id UUID,                   -- groups recs from same batch

  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_pricing_rec_s_t_s
ON pricing_recommendations(tenant_id, status, created_at DESC);

```

```
CREATE INDEX idx_pricing_recs_vehicle
  ON pricing_recommendations(vehicle_id, status);
```

pricing_recommendation_outcomes

Measured 14 days after `applied_at`.

```
CREATE TABLE pricing_recommendation_outcomes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  recommendation_id UUID NOT NULL UNIQUE
    REFERENCES pricing_recommendations(id) ON DELETE CASCADE,
  tenant_id UUID NOT NULL,
  vehicle_id UUID NOT NULL,

  measured_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  measurement_window_days INT NOT NULL DEFAULT 14,

  bookings_before INT,
  bookings_after INT,
  revenue_before NUMERIC(12,2),
  revenue_after NUMERIC(12,2),
  utilization_before NUMERIC(5,2),
  utilization_after NUMERIC(5,2),

  net_revenue_delta NUMERIC(10,2),
  outcome TEXT NOT NULL
    CHECK (outcome IN ('positive', 'neutral', 'negative')),

  created_at TIMESTAMPTZ DEFAULT now()
);
```

pricing_change_history

Full audit log. Every change to `vehicles.*_rent` goes here.

```
CREATE TABLE pricing_change_history (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL,
  vehicle_id UUID NOT NULL,
  tier TEXT NOT NULL,
  old_price NUMERIC(10,2),
  new_price NUMERIC(10,2) NOT NULL,
  change_source TEXT NOT NULL
    CHECK (change_source IN ('manual', 'ai_recommendation', 'autopilot', 'revert', 'import')),
  recommendation_id UUID REFERENCES pricing_recommendations(id),
  changed_by UUID REFERENCES app_users(id),
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_pch_vehicle ON pricing_change_history(vehicle_id, created_at DESC);
CREATE INDEX idx_pch_tenant ON pricing_change_history(tenant_id, created_at DESC);
```

revenue_optimiser_rules

Per-vehicle or per-category bounds for autopilot.

```
CREATE TABLE revenue_optimiser_rules (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,  
  vehicle_id UUID REFERENCES vehicles(id) ON DELETE CASCADE,  
  category TEXT,    -- e.g. 'economy', 'suv', 'luxury'  
  
  autopilot_enabled BOOLEAN DEFAULT FALSE,  
  
  min_price_daily NUMERIC(10,2),  
  max_price_daily NUMERIC(10,2),  
  min_price_weekly NUMERIC(10,2),  
  max_price_weekly NUMERIC(10,2),  
  min_price_monthly NUMERIC(10,2),  
  max_price_monthly NUMERIC(10,2),  
  
  created_at TIMESTAMPTZ DEFAULT now(),  
  updated_at TIMESTAMPTZ DEFAULT now(),  
  
  CONSTRAINT one_scope_only CHECK (  
    (vehicle_id IS NOT NULL AND category IS NULL) OR  
    (vehicle_id IS NULL AND category IS NOT NULL)  
  )  
);
```

pricing_experiments

A/B tests (Phase 3).

```
CREATE TABLE pricing_experiments (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id UUID NOT NULL,  
  vehicle_id UUID NOT NULL,  
  tier TEXT NOT NULL,  
  control_price NUMERIC(10,2) NOT NULL,  
  test_price NUMERIC(10,2) NOT NULL,  
  started_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  ends_at TIMESTAMPTZ NOT NULL,  
  control_bookings INT DEFAULT 0,  
  test_bookings INT DEFAULT 0,  
  control_revenue NUMERIC(12,2) DEFAULT 0,  
  test_revenue NUMERIC(12,2) DEFAULT 0,  
  winner TEXT CHECK (winner IN ('control', 'test', 'inconclusive')),  
  status TEXT NOT NULL DEFAULT 'running'  
    CHECK (status IN ('running', 'completed', 'aborted')),  
  created_at TIMESTAMPTZ DEFAULT now(),  
  updated_at TIMESTAMPTZ DEFAULT now()  
);
```

backtest_results

```
CREATE TABLE backtest_results (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,  
  period_start DATE NOT NULL,  
  period_end DATE NOT NULL,  
  actual_revenue NUMERIC(12,2) NOT NULL,  
  projected_revenue NUMERIC(12,2) NOT NULL,  
  uplift_percent NUMERIC(5,2) NOT NULL,  
  uplift_amount NUMERIC(12,2) NOT NULL,  
  vehicles_analysed INT,  
  bookings_analysed INT,  
  confidence TEXT NOT NULL,  
  per_vehicle_summary JSONB,  
  monthly_breakdown JSONB,  
  generated_at TIMESTAMPTZ DEFAULT now()  
);
```

9.2 Materialised view: vehicle_pricing_stats

Refreshed hourly via cron. The single source of truth for the recommendation engine.

```

CREATE MATERIALIZED VIEW vehicle_pricing_stats AS
SELECT
  v.id AS vehicle_id,
  v.tenant_id,
  v.daily_rent,
  v.weekly_rent,
  v.monthly_rent,
  -- 30-day window
  COUNT(DISTINCT r30.id) AS bookings_30d,
  COALESCE(SUM(r30.total_price), 0) AS revenue_30d,
  -- 90-day window
  COUNT(DISTINCT r90.id) AS bookings_90d,
  COALESCE(SUM(r90.total_price), 0) AS revenue_90d,
  -- utilisation
  CASE WHEN 30 > 0 THEN
    100.0 * COUNT(DISTINCT booked_days_30) / 30
  END AS utilization_30d,
  -- idle days
  (now()::date - MAX(r_all.end_date)) AS idle_days,
  -- enquiry signals
  (SELECT COUNT(*) FROM enquiries e
   WHERE e.vehicle_id = v.id
        AND e.created_at > now() - INTERVAL '14 days'
        AND e.status IN ('new', 'contacted')) AS active_enquiries_14d,
  -- conversion
  (SELECT COUNT(*) FILTER (WHERE e.converted_to_customer_id IS NOT NULL)::FLOAT /
   NULLIF(COUNT(*), 0)
   FROM enquiries e
   WHERE e.vehicle_id = v.id
        AND e.created_at > now() - INTERVAL '90 days') AS enquiry_conversion_90d,
  now() AS computed_at
FROM vehicles v
LEFT JOIN rentals r30 ON r30.vehicle_id = v.id
  AND r30.start_date > now() - INTERVAL '30 days'
  AND r30.status NOT IN ('cancelled', 'draft')
LEFT JOIN rentals r90 ON r90.vehicle_id = v.id
  AND r90.start_date > now() - INTERVAL '90 days'
  AND r90.status NOT IN ('cancelled', 'draft')
LEFT JOIN rentals r_all ON r_all.vehicle_id = v.id
  AND r_all.status = 'completed'
GROUP BY v.id, v.tenant_id, v.daily_rent, v.weekly_rent, v.monthly_rent;

CREATE UNIQUE INDEX ON vehicle_pricing_stats(vehicle_id);

```

The actual implementation will use day-bucketed CTEs for accurate utilisation; the above is illustrative.

9.3 RLS policies

All new tables follow the standard pattern.


```

-- read own tenant
CREATE POLICY tenant_read ON pricing_recommendations
  FOR SELECT USING (tenant_id = get_user_tenant_id() OR is_super_admin());

-- mutations only through service_role / edge functions
CREATE POLICY service_only ON pricing_recommendations
  FOR ALL USING (auth.role() = 'service_role');

```

9.4 Manager permissions

Add a new tab key to `src/lib/permissions.ts`:

```
'revenue_optimiser' – Revenue Optimiser tab (viewer/editor)
```

- Viewer: read recommendations, see outcomes, see backtest
- Editor: apply, dismiss, snooze, edit rules, toggle autopilot

10. Edge Functions

All Deno-based, following project conventions (shared CORS helper, JWT verified unless noted).

Function	Schedule	Purpose
<code>revenue-optimiser-refresh-stats</code>	Cron hourly	Refresh <code>vehicle_pricing_stats</code> materialised view
<code>revenue-optimiser-generate</code>	Cron daily (7am tenant TZ)	Generate recommendations for each enabled tenant
<code>revenue-optimiser-backtest</code>	On-demand	Run a backtest for a tenant; write to <code>backtest_results</code>
<code>revenue-optimiser-apply</code>	JWT	Apply a recommendation: update <code>vehicles</code> , write audit row
<code>revenue-optimiser-dismiss</code>	JWT	Mark dismissed with reason
<code>revenue-optimiser-snooze</code>	JWT	Snooze for N days
<code>revenue-optimiser-revert</code>	JWT	Revert an applied recommendation
<code>revenue-optimiser-measure-outcomes</code>	Cron daily	Find recs applied 14+ days ago, compute outcomes

Function	Schedule	Purpose
revenue-optimiser-autopilot-run	Cron daily	For autopilot tenants, auto-apply within bounds
revenue-optimiser-send-offers	JWT	Phase 4: send offers to matching enquiries
revenue-optimiser-daily-email	Cron daily	Email summary to each enabled tenant
revenue-optimiser-anomaly-check	Cron 6-hourly	Detect utilisation drops after applies → auto-pause

Function signatures (illustrative)

```
// revenue-optimiser-apply
Deno.serve(async (req) => {
  const { recommendationId, customPrice } = await req.json();
  // 1. fetch recommendation, verify tenant_id matches caller
  // 2. validate price within rules
  // 3. update vehicles.{tier}_rent
  // 4. insert pricing_change_history row
  // 5. update recommendation status='applied', applied_at, applied_price
  // 6. schedule outcome measurement (cron picks it up)
  // 7. return updated recommendation
});
```

11. Algorithm Specification

11.1 Pipeline

1. Refresh vehicle_pricing_stats (hourly)
2. For each enabled tenant (daily):
 - a. Identify eligible vehicles
 - has 60+ days of history, OR
 - is in tenant fleet ≥ 30 days
 - b. For each eligible vehicle:
 - i. Compute elasticity from historical price/bookings
 - ii. Compute Demand Score (0-100)
 - iii. Compute Supply Score (0-100)
 - iv. Compute Timing Score (0-100)
 - v. Compute recommended price band
 - vi. Compute confidence score
 - vii. Filter by minimum impact threshold (skip if $< \$30/\text{mo}$ delta)
 - viii. Apply safety rails (cost floor, max swing)
 - ix. Generate GPT explanation
 - x. Insert pricing_recommendations row
 - c. Mark prior open recommendations for same vehicle/tier as 'superseded'
3. Email daily summary

11.2 Price elasticity (own-fleet only)

For each vehicle, bucket bookings from the last 180 days by price:

```
For prices  $P_1, P_2, \dots, P_n$  observed in last 180 days:  
   $Q(P_i)$  = number of completed bookings at  $P_i$   
   $\text{Conversion}(P_i)$  = bookings at  $P_i$  / enquiries at  $P_i$  (if available)  
  
Fit log-log regression:  $\log(Q) = a + b \cdot \log(P)$   
   $\rightarrow$  elasticity  $\varepsilon = b$  (typically negative)  
  
Optimal price = argmax over  $P$  of:  $P \times \hat{Q}(P)$   
where  $\hat{Q}(P)$  is the fitted demand curve.  
  
Bounds:  
  recommended  $\in [P_{\text{current}} \times (1 - \text{max\_swing}), P_{\text{current}} \times (1 + \text{max\_swing})]$   
  recommended  $\geq \text{cost\_floor}$  (if enabled)
```

If a vehicle has fewer than 6 distinct price points or fewer than 12 bookings, fall back to fleet-level elasticity for the same vehicle category.

11.3 Demand Score (0–100)

```
demand_score =  
    0.30 × normalize(active_enquiries_14d)  
+ 0.25 × normalize(booking_velocity_trend)  
+ 0.25 × normalize(utilization_30d)  
+ 0.20 × (1 - normalize(idle_days))
```

Each input normalised against the tenant's own fleet (z-score → 0–100). This makes the score meaningful for both small and large fleets.

11.4 Supply Score (0–100)

```
supply_score =  
    0.40 × (1 - similar_vehicles_available_pct)  
+ 0.30 × utilization_30d  
+ 0.30 × (1 - days_until_next_booking_normalized)
```

High supply score = low availability = price can rise.

11.5 Timing Score (0–100)

```
timing_score = base_timing  
    + weekend_bonus (if recommendation period covers a weekend)  
    + holiday_bonus (if covers a tenant_holiday)  
    + lead_time_factor (last-minute demand → higher)
```

11.6 Final recommended price

```
demand_multiplier = (demand_score - 50) / 500    // ±10%  
supply_multiplier = (supply_score - 50) / 500    // ±10%  
timing_multiplier = (timing_score - 50) / 1000    // ±5%  
  
raw_recommended = optimal_price_from_elasticity  
    × (1 + demand_multiplier + supply_multiplier + timing_multiplier)  
  
recommended = clamp(raw_recommended,  
    current_price × (1 - max_swing),  
    current_price × (1 + max_swing))  
recommended = max(recommended, cost_floor)  
  
range_low = recommended × 0.97  
range_high = recommended × 1.05
```

11.7 Confidence score

```
confidence_score =  
    40 × min(1, bookings_90d / 30)           // sample size  
+ 30 × elasticity_r_squared                   // model fit  
+ 20 × (1 - normalized_variance_in_conversion)  
+ 10 × (1 if vehicle_age_in_fleet > 90d else 0)  
  
confidence_label =  
    HIGH    if score >= 70  
    MEDIUM if 40 <= score < 70  
    LOW     if score < 40
```

LOW confidence recommendations are shown but visually de-emphasised; autopilot ignores them.

11.8 Minimum impact threshold

Skip recommendations where projected monthly delta < \$30. This stops trivial noise.

12. AI Integration

GPT's role is **explanation only**. Never inference of price.

12.1 Where GPT is used

- Generate the plain-English explanation for each recommendation
- Generate the daily summary email body (optional — can use templates)
- Generate the backtest narrative copy

12.2 Model

- `gpt-4o-mini` for explanations (~\$0.15 / 1M tokens input).
- Avoid `gpt-4o` for routine work — only use for backtest narratives or low-volume calls.

12.3 Prompt template (explanation)

SYSTEM:

You are Drive247 Revenue Optimiser's explanation writer. You receive a JSON payload describing a pricing recommendation that was computed by a deterministic statistical model. Your job is to write a short, factual, plain-English explanation. Do NOT invent prices, numbers, or trends not present in the payload. Do NOT use marketing language. Be specific. 2–3 sentences. Reference the most important 2 data points.

USER:

```
{
  "vehicle": "Toyota Corolla ABC-123",
  "tier": "weekly",
  "current_price": 300,
  "recommended_price": 325,
  "data_points": {
    "utilization_30d": 0.93,
    "active_enquiries_14d": 4,
    "conversion_at_current": 0.76,
    "bookings_velocity_trend_14d": 0.08,
    "fleet_avg_utilization": 0.78
  },
  "reasons_top_3": ["high_utilization", "active_demand", "conversion_strong"]
}
```

12.4 Cost controls

- Pre-compute everything statistically before calling GPT
- Cache identical explanations across vehicles when reasons match
- Skip GPT if explanation cache hit (vehicle category × reason set)
- Log every call to `openai_usage_logs` for per-tenant cost tracking

12.5 Hard fallback

If GPT call fails, generate a template-based explanation from the reasons array. Recommendation still ships; only the prose differs.

13. Safety & Trust Mechanisms

A recommendation is only shipped to the operator after passing all of these:

1. **Sample size gate** — vehicle must have ≥ 12 bookings in 90 days OR fall back to category model
2. **Calibration gate** — tenant must have ≥ 30 days of data
3. **Cost floor** — never below tenant-set floor
4. **Max swing** — never exceeds tenant-set max swing %

5. **Stale lock** — if a recommendation for this vehicle/tier was applied in last 14 days, no new one (let outcome land first)
6. **Approval threshold** — if change exceeds operator's approval threshold, recommendation is flagged "needs approval" rather than auto-applied even under autopilot
7. **Outcome dependency** — autopilot pauses for a vehicle if its last 2 outcomes were negative
8. **Utilisation circuit breaker** — autopilot pauses fleet-wide if utilisation drops >20% within 7 days of any apply
9. **Anomaly detector** — `revenue-optimiser-anomaly-check` runs every 6 hours, flags weird recommendations (e.g., > +25% swing) for super-admin review
10. **Audit log immutable** — `pricing_change_history` rows are insert-only, retained 24 months

14. What to Show / What to Hide

Surface	Show to operator	Show to super-admin	Hide entirely
Recommendation price	✓	✓	—
Reasons (structured)	✓	✓	—
Data points behind reasons	✓	✓	—
Confidence label + score	✓ (label only)	✓ (full)	—
Elasticity curve	✓	✓	—
GPT-generated prose	✓	✓	—
Internal multiplier coefficients	✗	✓	—
Raw GPT prompt	✗	✓	—
Other tenants' data	✗	✓ (anonymised)	—
OpenAI cost per call	✗	✓	—
Per-tenant model accuracy	✓ (own only)	✓ (all)	—
Reverted recommendations	✓ (own history)	✓	—
Single-vehicle data with sample < 12	"Not enough data" message	✓	—

Principle: show everything that supports the decision; hide the sausage-making.

15. Monitoring & Telemetry

Per-tenant dashboard (admin)

Tenant: City Wheels

Optimiser mode:

Recommendations

Calibration done:

✓ (12 weeks ago)

Apply rate:

93% (38 of 41)

Positive outcome:

87%

Measured uplift:

+\$4,820 (90d)

Recommendation accuracy trend:

▲ improving

OpenAI cost (30d):

\$4.18

Last anomaly:

—

System health (admin)

- Daily recommendations generated count
- Edge function error rate
- GPT call latency p50/p95/p99
- Outcome measurement completeness (% of applied recs with measured outcome)
- Aggregate uplift across all tenants

Anomaly inbox (admin)

- Recommendations swinging > 20%
- Tenants with sudden mode changes
- Apply-then-revert within 24h (operator distrust signal)
- Auto-pause triggers

16. Subscription Tier & Pricing

Tier placement

Revenue Optimiser sits in a new **Growth** tier (or whatever the existing premium tier is named in `subscription_plans`). Suggested structure:

Tier	Has Revenue Optimiser?
Basic	No
Pro	Insights only (read)
Growth	Recommendations + Autopilot

Why charge for it

OpenAI cost is real, but small. The price tag exists because:

1. It anchors the value as premium
2. It filters for serious operators (the ones who'll actually apply recommendations)
3. It funds ongoing improvement of the elasticity model

Cost model (rough)

- ~50 vehicles × 1 recommendation/day × 800 tokens explanation × gpt-4o-mini
- ≈ 40k tokens/day per tenant ≈ \$0.01/day ≈ \$0.30/month in OpenAI cost
- Add backtest (1× setup ~\$0.40) + on-demand backtests (rare)
- Total per-tenant cost: \$0.30–\$0.50/month

A \$39–\$99/month tier upgrade easily covers this with margin.

17. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Bad recommendation loses tenant money	Medium	High	Safety rails, calibration period, outcome measurement, revert button
Tenant doesn't trust the feature	High	High	Backtest first, show your work, outcome tracking
Data sparsity for small fleets	High	Medium	Calibration gate, fall back to category models, "not enough data" UI
OpenAI cost spikes	Low	Low	Cached explanations, cost caps, fallback to templates
Autopilot creates a feedback loop	Low	High	Anomaly detector, utilisation circuit breaker, A/B testing
Operator over-relies on autopilot	Medium	Medium	Weekly digest highlighting changes, easy disable
Compliance / pricing transparency	Low	Medium	Audit log, show change history to customers if required
Competitor copies the feature	High	Medium	Outcome data is the moat; first-mover trust matters most

18. Success Metrics

Measure ruthlessly. Cut what doesn't work.

Activation

- % of eligible tenants who run backtest within 7 days of seeing the feature: target 70%
- % who enable Insights mode after seeing backtest: target 50%
- % who progress from Insights to Recommendations within 30 days: target 40%

Engagement

- Daily active operators on Revenue Optimiser page: track
- Recommendation apply rate (applied / shown): target 35%+
- Recommendation dismiss rate: monitor; <40% is healthy

Outcome

- Median measured revenue uplift per tenant after 90 days: target +5%
- % of applied recommendations with positive outcome: target 70%+
- Autopilot enablement: target 25% of Growth tenants by day 180

Business

- Growth tier conversion lift: target +20%
 - Retention impact: target +5pp 12-month retention on Growth vs. Pro
 - NPS impact: +10 points
-

19. Build Checklist

Each phase has acceptance criteria. Don't progress until checked.

Phase 0 — Foundation (Week 1)

- [] Migration: `revenue_optimiser_settings` , `pricing_change_history` , `backtest_results`
- [] Materialised view: `vehicle_pricing_stats` (with hourly refresh cron)
- [] Backtest engine (`revenue-optimiser-backtest` edge function)
- [] Data quality validation (skip vehicles with malformed data)
- [] Admin tool: trigger backtest for any tenant on demand
- [] Cost-floor input on vehicle edit page (operator sets per-vehicle break-even)

Acceptance: Backtest runs end-to-end for at least 3 test tenants. Output PDF generated.

Phase 1 — Insights (Weeks 2–3)

- [] Sidebar entry “Revenue Optimiser” with “NEW” badge
- [] Welcome screen with “Run Backtest” CTA
- [] Backtest report screen with chart + per-vehicle table
- [] Insights Mode: daily aggregation cron writes observations (no recommendations yet)
- [] Calibration logic: new tenants in observation for 30 days
- [] Manager permission key `revenue_optimiser` (viewer/editor)
- [] Subscription tier gating

Acceptance: 3 internal test tenants enable Insights Mode and see at least 1 useful observation/day for a week.

Phase 2 — Recommendations (Weeks 4–5)

- [] Migration: `pricing_recommendations`, `pricing_recommendation_outcomes`
- [] Edge function: `revenue-optimiser-generate` (daily cron)
- [] Elasticity model implementation (TypeScript or SQL-side)
- [] Demand/Supply/Timing scoring
- [] Confidence calculation
- [] GPT explanation generation with caching
- [] Recommendations list page
- [] Recommendation detail drawer with elasticity curve
- [] Vehicle detail Smart Pricing panel
- [] Apply / Dismiss / Snooze / Custom flows
- [] Outcome measurement cron + outcomes table population
- [] Outcomes screen
- [] Daily summary email
- [] Audit log writes wired through every change
- [] All safety rails enforced

Acceptance: 5 paying tenants run Phase 2 for 30 days. Median measured uplift positive. Apply rate $\geq$ 30%.

Phase 3 — Rules & Autopilot (Weeks 6–7)

- [] Migration: `revenue_optimiser_rules`, `pricing_experiments`
- [] Rules editor UI (per-category, per-vehicle)
- [] Autopilot configuration wizard
- [] Autopilot daily run edge function
- [] A/B testing framework (control vs. test group, 14d window)
- [] Anomaly detector + auto-pause logic
- [] Outcome-based vehicle pausing (2 negatives $\rightarrow$ pause)

- ☐ Approval-threshold gating
- ☐ Anomaly inbox in admin

Acceptance: Autopilot opt-in by 2 Growth tenants, running 30 days with no fleet-wide circuit breaker triggered, measured positive uplift.

Phase 4 — Lead Hub Integration (Week 8)

- ☐ Match enquiries to vehicle + date window
- ☐ Combined recommendation card (price + send offer)
- ☐ Offer preview screen with editable message
- ☐ Send via existing email/SMS infrastructure
- ☐ Track offer-to-booking conversion
- ☐ Outcomes screen now includes offer engagement

Acceptance: First offer flow used by a paying tenant resulting in a booking traceable end-to-end.

Cross-cutting

- ☐ Documentation: operator help center article
 - ☐ Documentation: internal runbook for support
 - ☐ Sales deck slide
 - ☐ Demo tenant set up for sales calls
 - ☐ Monitoring dashboards in admin
 - ☐ Anomaly alert wiring to Slack/email for Drive247 ops
 - ☐ OpenAI cost tracking visible per tenant
-

20. Appendix

A. Glossary

- **Elasticity** — how booking quantity changes with price. A value of -1 means a 10% price increase causes a 10% drop in bookings.
- **Calibration** — initial period (30 days) during which the engine collects data and makes no recommendations.
- **Autopilot** — automatic application of recommendations within tenant-defined rules.
- **Backtest** — replaying historical bookings against what the engine would have recommended.
- **Outcome** — measured net impact of an applied recommendation, taken 14 days after apply.
- **Lead Hub** — Drive247's enquiry management system (the existing `enquiries` table).

B. Reasons taxonomy (for `pricing_recommendations.reasons`)

Stable codes used by both UI rendering and outcome attribution.

Code	Label	Trigger
<code>high_utilization</code>	High utilisation	<code>utilization_30d > fleet avg + 10pp</code>
<code>low_utilization</code>	Low utilisation	<code>utilization_30d < fleet avg - 10pp</code>
<code>idle_streak</code>	Idle vehicle	<code>idle_days ≥ 5</code>
<code>active_demand</code>	Active enquiries	<code>active_enquiries_14d ≥ 3</code>
<code>weekend_pickup</code>	Weekend uplift	<code>period covers weekend</code>
<code>holiday_period</code>	Holiday surcharge available	<code>period covers <code>tenant_holidays</code> row</code>
<code>conversion_strong</code>	Strong conversion	<code>conversion at current price ≥ 70%</code>
<code>conversion_weak</code>	Weak conversion	<code>conversion at current price ≤ 40%</code>
<code>velocity_up</code>	Booking velocity rising	<code>velocity_trend_14d > +5%</code>
<code>velocity_down</code>	Booking velocity falling	<code>velocity_trend_14d < -5%</code>
<code>competitive_idle</code>	Similar cars all booked	<code>similar_available_pct < 25%</code>
<code>fleet_supply_high</code>	Fleet supply abundant	<code>similar_available_pct > 75%</code>

C. Example structured recommendation row

```
{
  "id": "f8e3...",
  "tenant_id": "1234...",
  "vehicle_id": "abcd...",
  "tier": "weekly",
  "current_price": 300.00,
  "recommended_price": 325.00,
  "recommended_range_low": 315.00,
  "recommended_range_high": 330.00,
  "confidence": "high",
  "confidence_score": 82,
  "projected_revenue_delta_monthly": 95,
  "reasons": [
    { "code": "high_utilization", "label": "93% booked (last 30d)", "weight": 0.35 },
    { "code": "active_demand", "label": "4 active enquiries", "weight": 0.30 },
    { "code": "conversion_strong", "label": "76% conversion at $300", "weight": 0.20 },
    { "code": "velocity_up", "label": "+8% bookings vs. prior 14d", "weight": 0.15 }
  ],
  "data_points": {
    "bookings_30d": 4,
    "bookings_90d": 12,
    "utilization_30d": 0.93,
    "idle_days": 2,
    "active_enquiries_14d": 4,
    "conversion_at_current_price": 0.76
  },
  "elasticity_curve": [
    { "price": 290, "predicted_bookings_per_month": 4.3 },
    { "price": 300, "predicted_bookings_per_month": 4.1 },
    { "price": 315, "predicted_bookings_per_month": 3.9 },
    { "price": 325, "predicted_bookings_per_month": 3.8 },
    { "price": 340, "predicted_bookings_per_month": 3.4 }
  ],
  "ai_explanation": "This vehicle is one of your most-rented. It's been booked 93% of the last",
  "status": "pending",
  "expires_at": "2026-06-06T00:00:00Z",
  "created_at": "2026-05-23T07:00:00Z"
}
```

D. Open questions to revisit before Phase 3

- Should we expose `cost_floor` editing inside Revenue Optimiser settings, or keep it on the vehicle edit page?
 - Should Phase 4 offers go via WhatsApp first (we have Twilio Content Templates) or email/SMS?
 - How do we handle multi-currency tenants in fleet-wide normalisation? (Probably stratify by currency.)
 - Should we surface aggregated insights from across all tenants (anonymised) to help small tenants benefit from network effects?
-

Document version: 1.0 — generated for Drive247 by the engineering team. Update as decisions evolve.